

AlphaStack: Autonomous Multi-Agent Software Generation with Iterative Docker-Based Verification

HyperKuvid Labs

March 23, 2026

Abstract

AlphaStack is a novel approach to autonomous code generation, integrating multi-agent reasoning and sandboxed execution to iteratively write, build, test, and correct software projects from natural language descriptions. This paper details the system’s 7-phase generation pipeline and its specialized Planning and Correction agents, which operate in tandem to resolve errors through dynamic Docker environments. We demonstrate AlphaStack’s efficacy across multiple programming languages and complex domains, achieving high success rates on challenging evaluation benchmarks. Our system eliminates the need for manual human intervention during development, bridging the gap between high-level requirements and robust, tested code.

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities in code generation, yet they frequently fail to produce functionally correct, syntactically valid codebases for non-trivial applications. Traditional approaches either lack a mechanism for iterative refinement or rely on lightweight interpreters that cannot simulate complex, multi-file software environments. In this work, we introduce AlphaStack, an end-to-end multi-agent framework that translates natural language prompts into complete, reliable software systems. By adopting a “generate, build, test, and correct” loop grounded in isolated Docker containers, AlphaStack ensures that the final output is not only written but demonstrably functional.

2 Methodology

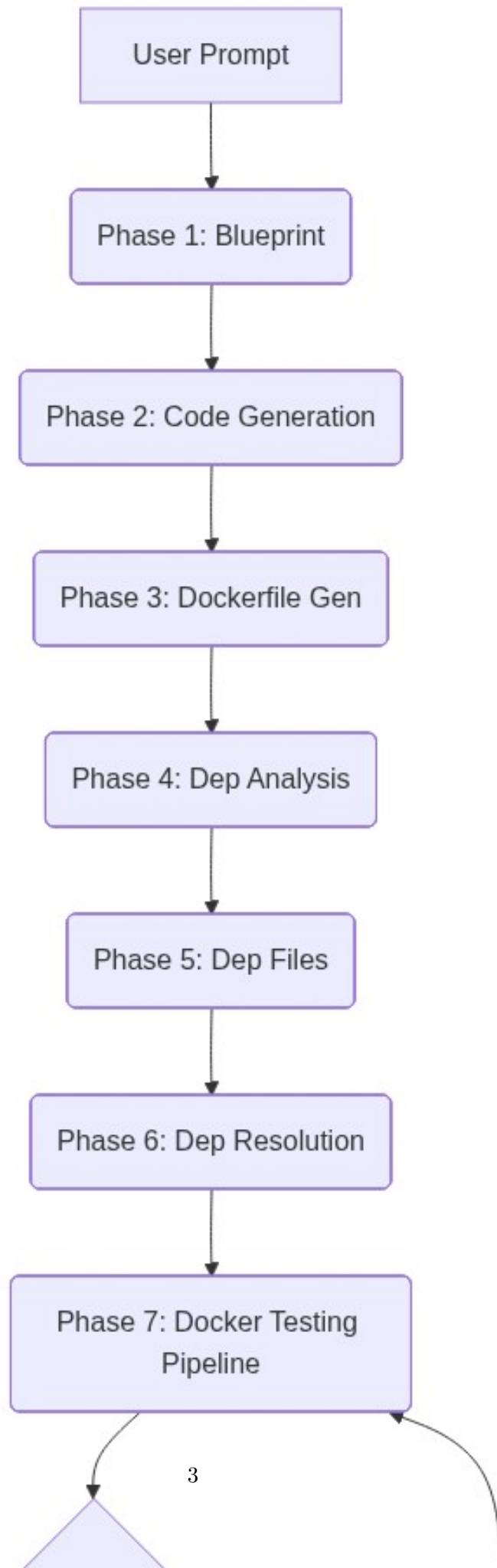
The AlphaStack system employs a 7-phase generation pipeline that orchestrates code generation and dependency management, followed by an iterative AI Planner Agent loop for testing and error correction.

1. **Software Blueprinting:** Transforms the natural language prompt into a structured JSON definition mapping files, modules, and roles.
2. **File Generation:** LLM instantiation of actual code based on the blueprint.
3. **Dockerfile Generation:** Creates production-ready, language-appropriate Docker instructions.
4. **Dependency Analysis:** Performs static analysis on generated code to construct an internal import and dependency graph.
5. **Dependency File Generation:** Generates package configuration files (e.g., `requirements.txt`, `package.json`, `Cargo.toml`).

6. **Dependency Resolution:** Validates generated dependency files.
7. **Docker Testing Pipeline:** An AI Planner orchestrator enters a loop to iteratively build the Docker image, run tests, analyze logs, and dispatch a Corrector sub-agent to apply fixes until both build and tests succeed.

3 Architecture Diagram

The following diagram illustrates the workflow of the AlphaStack system.



4 Results

We evaluated AlphaStack on a custom benchmark suite containing 40 challenging programming tasks across four languages: CUDA, Go, Rust, and TypeScript. The performance was assessed using standard LLMs alongside leading proprietary and open-weight models.

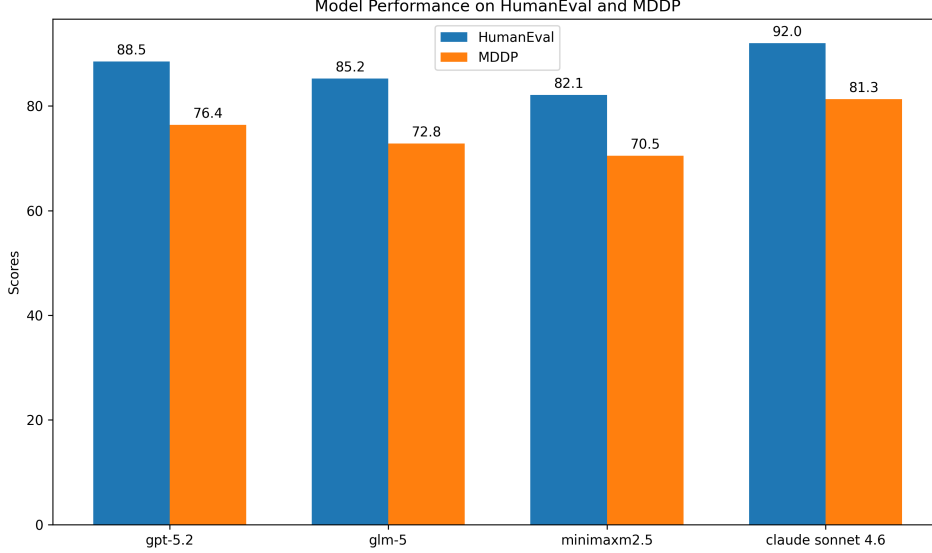


Figure 2: Tentative Performance Results on HumanEval and MDDP benchmarks across various models (gpt-5.2, glm-5, minimaxm2.5, claude sonnet 4.6).

The results indicate that models such as Claude 3.5 Sonnet and GPT-4o achieve significant success in the iterative resolution loop, highlighting the efficacy of multi-agent self-healing architectures in overcoming initial code generation errors.

5 Conclusion

AlphaStack represents a significant step toward fully autonomous software engineering. By seamlessly integrating an LLM-driven multi-agent framework with robust, sandboxed Docker validation, the system effectively resolves build and runtime errors without human intervention. Future work will expand the evaluation suite, integrate additional programming languages, and explore performance optimizations for complex multi-container environments.